

20: Parallelization (June 4th, 2026)

Lecturer: Hao Zhang

Scribe: Z. Wang, S. Ge, Y. Wang, Q. Zhang, Y. Zhang, J. Hou

1 Inter-Operator Parallelism

Inter-operator parallelism partitions a neural network computational graph into multiple **stages**, and places different stages on different devices. For example, with four devices, stage 1 runs on device 1, stage 2 runs on device 2, and so on.

During execution, the input first goes through stage 1. The output of stage 1 is sent to stage 2, the output of stage 2 is sent to stage 3, and finally stage 4 produces the model output. This data movement is usually point-to-point communication between adjacent stages, so it is cheaper than collective communication.

1.1 Pipeline Bubbles

A pipeline schedule is usually visualized with time on the x -axis and devices on the y -axis. Colored blocks are computation, and gray areas are idle time, called **pipeline bubbles**.

In the naive schedule, only one device is active at a time, because each stage depends on the previous stage's output. With D devices, the pipeline bubble percentage is

$$\frac{\text{bubble area}}{\text{total area}} = \frac{D-1}{D}.$$

When D is large, this is close to 1, meaning most device time is idle.

1.2 Pipelining Inputs

The main way to reduce bubbles is to pipeline multiple inputs. Once input a moves from stage 1 to stage 2, stage 1 can immediately process input b . After the pipeline is filled, different devices process different inputs at the same time.

With D devices and N inputs, the bubble percentage becomes

$$\frac{D-1}{D-1+N}.$$

This works well for inference, where many batches can continuously enter the pipeline. In pipeline parallelism, these small units are usually called **micro-batches**.

1.3 Training Dependency

Training is harder because it has both forward and backward passes. The forward pass goes from early stages to later stages, while the backward pass goes in the reverse direction. After gradients are computed,

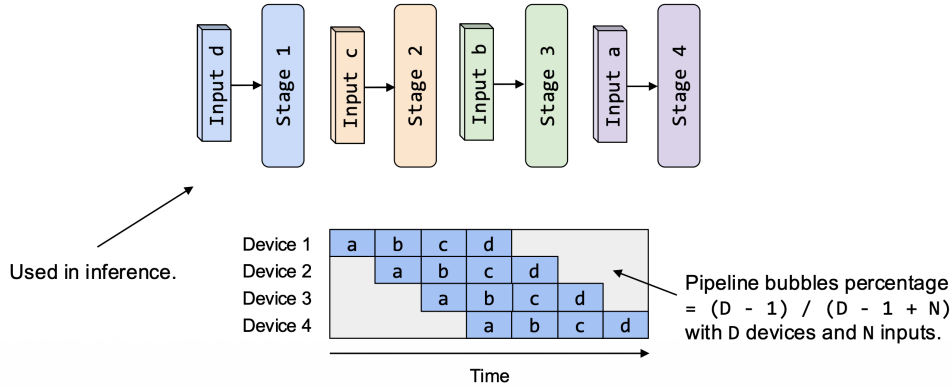


Figure 1: Pipelining multiple inputs reduces bubbles after warm-up.

the weights are updated.

Thus, a naive training schedule still has many bubbles: it runs forward through all devices, then backward through all devices, and finally updates the weights. The lecture discusses three ways to reduce training bubbles: device placement, synchronous pipeline parallel algorithms and asynchronous pipeline parallel algorithms.

1.4 Device Placement

Device placement slices independent branches of a neural network into different stages so that they can run concurrently. This can help for models with explicit branches, such as Inception modules or contrastive models.

However, it does not work well for mostly sequential models such as Transformers. Also, device utilization is still low, so device placement usually needs to be combined with other pipeline schedules.

2 Synchronous Pipeline Parallel Schedule

Synchronous pipeline schedules improve utilization while keeping the same computation and convergence semantics as single-device training.

2.1 GPipe

GPipe splits one input batch into multiple **micro-batches**, pipelines them through the stages, accumulates their gradients, and updates the model once. If x is the original batch and x_i is the i -th micro-batch, then

$$\nabla L_{\theta}(x) = \frac{1}{N} \sum_{i=1}^N \nabla L_{\theta}(x_i).$$

This is the same idea as gradient accumulation.

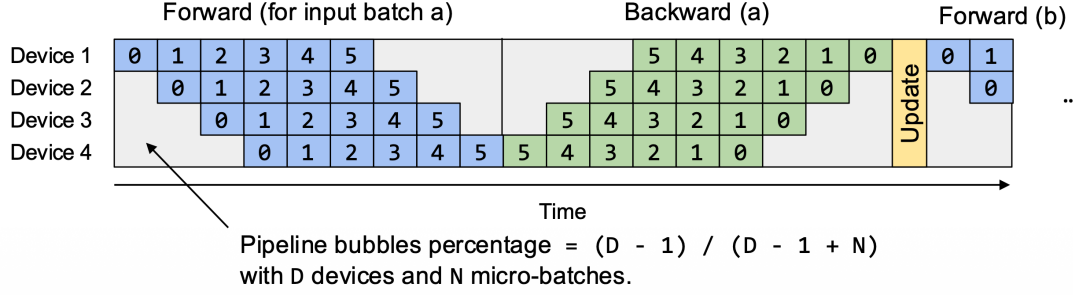


Figure 2: GPipe runs all forward micro-batches first, then all backward micro-batches, and finally updates the model.

GPipe: Memory Usage

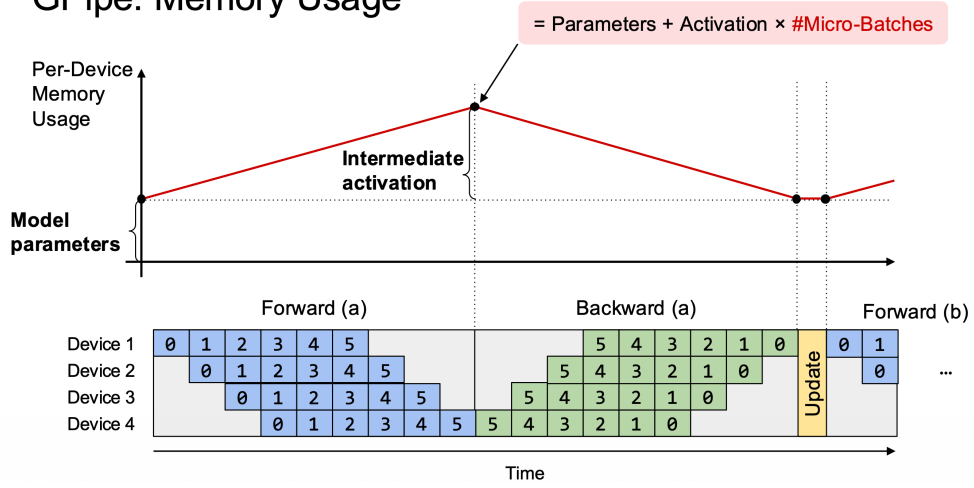


Figure 3: GPipe activation memory grows with the number of micro-batches.

GPipe respects the training dependency: a micro-batch can only start backward after its forward pass finishes, and backward computation follows the reverse stage order. Its bubble percentage is

$$\frac{D - 1}{D - 1 + N},$$

where D is the number of devices and N is the number of micro-batches. More micro-batches reduce bubbles and improve pipeline efficiency.

2.2 GPipe Memory Usage

The main issue of GPipe is memory. Since GPipe performs all forward passes before starting backward, each device must store intermediate activations for all micro-batches.

The peak per-device memory usage is approximately

$$\text{Parameters} + \text{Activation} \times \# \text{Micro-Batches}.$$

Therefore, GPipe has a trade-off: more micro-batches reduce bubbles, but also increase activation memory linearly. This memory problem motivates the 1F1B schedule, which performs backward as early as possible.

GPipe Schedule:

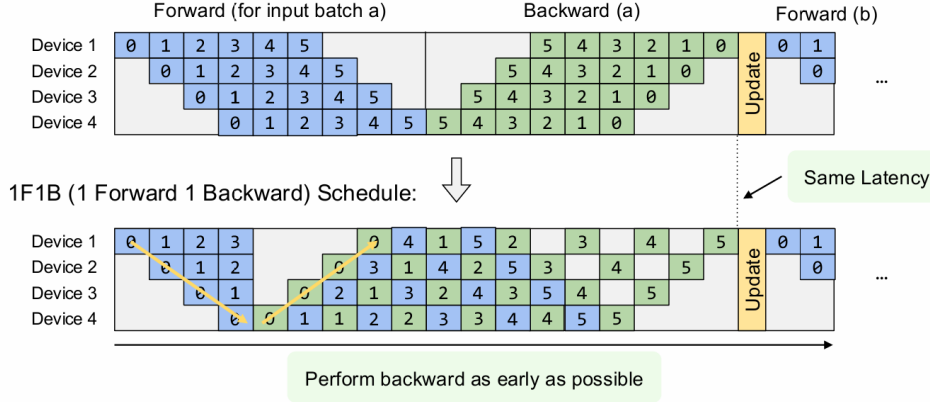


Figure 4: Comparison between the GPipe schedule and the 1F1B schedule. 1F1B reorders the forward and backward blocks and performs backward as early as possible while keeping the same overall training semantics.

2.3 1F1B Pipeline Schedule

GPipe’s memory cost grows linearly with the number of micro-batches, since all forward activations must be retained until the backward phase begins. The 1F1B (one forward, one backward) schedule addresses this by performing backward computation as early as possible: once a micro-batch finishes its forward pass on the last stage, its backward pass begins immediately rather than waiting for all other micro-batches to complete their forward passes.

This schedule can be viewed as a reordering of the forward and backward blocks in GPipe, as shown in Figure 4. It does not change the computation semantics. The important difference is the order in which the backward computations are scheduled. For example, for micro-batch 0, after it finishes the forward pass on the last device, its backward pass can start immediately. We do not need to wait until all other micro-batches finish their forward passes.

The dependency is still correct. A backward pass for a micro-batch cannot start before its forward pass finishes. Also, the backward pass must follow the reverse stage dependency: the backward computation on stage i depends on the gradient from stage $i + 1$. 1F1B respects both constraints. It only moves backward computations earlier when these dependencies have already been satisfied.

The benefit is that the activation lifetime becomes shorter. In GPipe, the activation of an early micro-batch may need to be stored until all forward passes finish. In 1F1B, as soon as the backward computation of that micro-batch finishes, its activation can be freed. Therefore, 1F1B greatly reduces peak activation memory.

2.4 Memory Usage of 1F1B

Let P be the memory used by the model parameters stored on a device, and let A be the activation memory for one micro-batch on that device. In GPipe, the peak per-device memory is approximately

$$P + A \times N,$$

where N is the number of micro-batches. This is because GPipe may need to keep activations for all micro-batches before it starts releasing them during backward computation.

In contrast, in 1F1B, the peak memory is approximately

$$P + A \times D,$$

where D is the number of pipeline devices or stages. The important improvement is that the activation factor changes from the number of micro-batches to the number of devices. Since the number of devices is fixed and usually much smaller than the number of micro-batches that we would like to use, this makes the schedule much more scalable.

The intuition is that after the pipeline is warmed up, every forward computation is paired with a nearby backward computation. When a backward computation finishes, the corresponding forward activation can be released. Therefore, the memory usage no longer keeps growing with the total number of micro-batches.

2.5 Interleaved 1F1B

A further optimization is **interleaved 1F1B**. The idea is to slice the neural network into more fine-grained pipeline stages and assign multiple stages to each device.

For example, suppose a model has eight layers and is trained on four devices. A simple pipeline partition may group the layers into four contiguous stages:

$$(L_1, L_2), \quad (L_3, L_4), \quad (L_5, L_6), \quad (L_7, L_8).$$

Each device then owns one contiguous stage. In interleaved 1F1B, we can instead make each layer its own stage and assign non-contiguous stages to the same device:

Device 1 : $L_1, L_5,$

Device 2 : $L_2, L_6,$

Device 3 : $L_3, L_7,$

Device 4 : $L_4, L_8.$

This makes each pipeline stage shorter. As a result, a later device does not need to wait as long before it can begin computation. From the timeline view, this gives the scheduler more fine-grained blocks to place into otherwise idle regions, reducing the bubble area.

If each device holds K pipeline stages and there are N micro-batches, the pipeline bubble percentage can be written as

$$\frac{D - 1}{D - 1 + KN},$$

where D is the number of devices. Compared with the non-interleaved case, the denominator contains KN instead of N , so the bubble percentage can be reduced.

However, since stages are more fine-grained and non-contiguous, activations must be transferred between devices more often. This increases communication overhead. In practice, whether interleaved 1F1B is better depends on the balance between reduced bubbles and increased communication. For large language model training, the standard 1F1B schedule is often attractive because it is simpler, and large batches can already be split into many micro-batches, making the original pipeline bubble small enough.

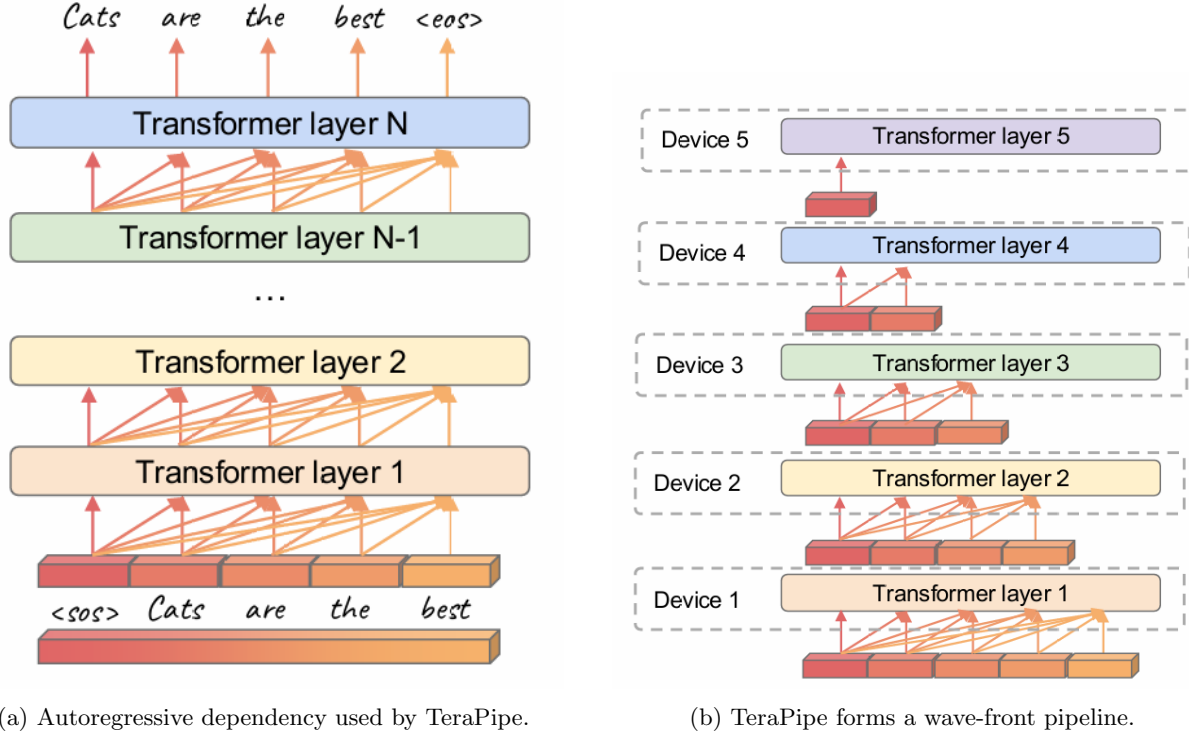


Figure 5: TeraPipe exploits autoregressive dependency to pipeline computation within a sequence. The computation of a token only depends on previous tokens, not future tokens, which enables wave-front pipeline parallelism across tokens and layers.

2.6 TeraPipe: Token-Level Pipeline Parallelism

The previous pipeline schedules split computation across layers and micro-batches. TeraPipe adds another dimension: it pipelines computation within a sequence. This is possible for autoregressive Transformer models.

TeraPipe uses this causal dependency structure to pipeline tokens through Transformer layers. At the first time step, device 1 can process the first token on layer 1. At the next time step, device 1 can process the second token on layer 1, while device 2 processes the first token on layer 2. This can continue across more layers and devices.

This forms a wave-front pipeline. The wave moves from earlier layers and earlier tokens toward later layers and later tokens. After the pipeline is filled, multiple devices can be busy at the same time, even within one input sequence. Therefore, TeraPipe can reduce bubbles by exploiting token-level parallelism in addition to layer-level pipeline parallelism.

The key condition is that this schedule must respect the autoregressive dependency. A token can depend on previous tokens, but it cannot depend on future tokens. Because of this property, token-level pipelining is valid for autoregressive language models.

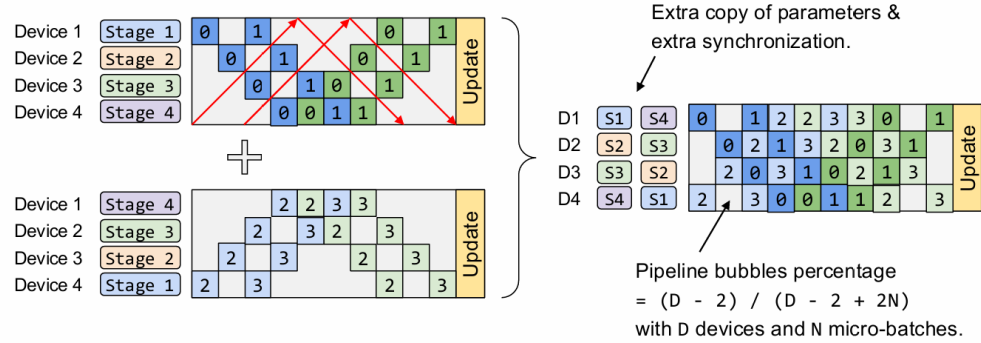


Figure 6: Chimera combines two pipelines in opposite directions. This reduces pipeline bubbles, but requires extra parameter copies and synchronization.

2.7 Chimera: Bidirectional Pipeline

Chimera is another optimization of 1F1B. The idea is to store bidirectional stages and combine two opposite-direction pipelines to further reduce pipeline bubbles.

Consider a pipeline with four stages. A normal pipeline maps stages in the forward direction, but Chimera also maintains a reverse pipeline.

If the two schedules are merged carefully, the empty regions in one schedule can be filled by computations from the other schedule. This makes the final timeline more saturated than either pipeline alone.

For Chimera, the bubble percentage shown in the lecture is

$$\frac{D - 2}{D - 2 + 2N},$$

where D is the number of devices and N is the number of micro-batches. The factor $2N$ reflects that two pipelines are being combined.

The downside is memory and communication. Since each device may need to store stages from both directions, it needs an extra copy of some parameters. This is different from the GPipe memory issue. In GPipe, the extra memory mainly comes from storing more activations. In Chimera, the extra memory comes from storing additional weights. Moreover, multiple copies of the same stage's weights may require extra synchronization and communication.

2.8 Asynchronous Pipeline Parallelism

The lecture also briefly discussed asynchronous pipeline schedules. The synchronous schedules above all preserve the same training semantics as single-device training: the next large batch is not scheduled before the previous large batch finishes and updates the weights.

Asynchronous pipeline parallelism tries to reduce bubbles more aggressively. It may start the next batch before the previous batch has fully finished. This means the next batch may use stale weights, because it does not wait for the most recent update. This can improve pipeline utilization, but it changes the semantics of training.

The main problem is convergence. Since asynchronous schedules do not behave exactly like standard single-device training, they can introduce optimization instability. The lecture noted that this line of work is less

important today because such convergence issues are usually not attractive for large-scale model training.

2.9 Short Summary

At this point, the lecture finishes the discussion of inter-op parallelism and moves to intra-op parallelism.

The main model for pipeline parallelism is a timeline where the x -axis is time and the y -axis is device. Computation blocks occupy parts of this timeline, while gray areas correspond to idle devices, or pipeline bubbles. When evaluating a pipeline schedule, one should ask how much bubble area remains.

The key schedules can be summarized as follows:

Schedule	Main idea	Main cost
GPipe	Pipeline many micro-batches	Activation memory grows with N
1F1B	Run backward as early as possible	More complex scheduling than GPipe
Interleaved 1F1B	Use more fine-grained stages	More communication between stages
TeraPipe	Pipeline within a sequence	Requires autoregressive dependency structure
Chimera	Merge forward and reverse pipelines	Extra parameter copies and synchronization

In inter-op parallelism, the main problem is pipeline bubbles because different devices may wait for dependencies across stages. In intra-op parallelism, devices are usually busy, so bubbles are not the main issue. Instead, the key challenge becomes communication, since partitioning a single operator often requires collective communication between devices.

3 Intra-Operator Parallelism and Graph-Level Parallelization

The previous part of the lecture discussed inter-operator parallelism, where a neural network is divided into stages and different stages are placed on different devices. In this section, we study a complementary form of parallelism: *intra-operator parallelism*. Instead of assigning different operators to different devices, intra-operator parallelism splits the computation *inside a single operator* across multiple devices.

At a high level, the key question changes from “which device should run which operator?” to “how should we partition the loops and tensors inside each operator?” This question is simple for element-wise operators, more subtle for matrix multiplication, and even more challenging when we need to choose compatible parallelization strategies for all operators in a computational graph.

3.1 From Inter-op to Intra-op Parallelism

Figure 7 illustrates the distinction between the two forms of parallelism. In inter-operator parallelism, different operators or stages in a graph are assigned to different devices. Pipeline parallelism is a representative example: early layers are placed on one set of devices, later layers are placed on another set of devices, and activations are transferred between stages.

In intra-operator parallelism, a single operator is split across devices. For example, a matrix multiplication can be partitioned by rows, columns, or the reduction dimension. Each partitioning strategy leads to a different memory layout and a different communication pattern. Therefore, intra-operator parallelism is not just about dividing work evenly; it is also about choosing tensor partitions that minimize communication.

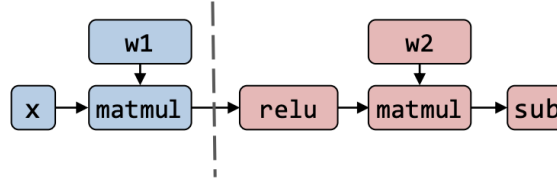
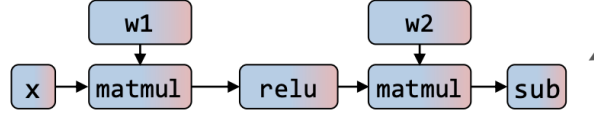
Strategy 1: Inter-operator Parallelism**Strategy 2: Intra-operator Parallelism**

Figure 7: Inter-operator parallelism partitions a computational graph across devices, while intra-operator parallelism partitions the computation within an individual operator.

This section follows three steps. First, we examine how to parallelize a single operator. Second, we explain why matrix multiplication may require collective communication. Third, we discuss how to choose parallelization strategies for an entire computational graph.

3.2 Parallelizing Element-wise Operators

We begin with the simplest case: an element-wise operator. Consider the element-wise addition

$$C[n, d] = A[n, d] + B[n, d]. \quad (1)$$

A loop-based implementation is

$$\begin{aligned} &\text{for } n = 0, \dots, N - 1 : \\ &\quad \text{for } d = 0, \dots, D - 1 : \\ &\quad\quad C[n, d] = A[n, d] + B[n, d]. \end{aligned} \quad (2)$$

The important property is that different output elements are independent. Computing $C[n, d]$ does not depend on $C[n', d']$ for any other pair (n', d') . Therefore, both loop dimensions n and d are *spatial loops*: they can be partitioned freely across devices.

Figure 8 shows several possible partitioning choices. We can split the tensor by rows, by columns, or by a two-dimensional tiling. Since each device can compute its own output shard independently, no communication is needed during the element-wise computation itself. This is why element-wise operators are often described as *embarrassingly parallel*.

However, most expensive neural network operators are not purely element-wise. Matrix multiplication, convolution, and attention all contain reduction dimensions. These reduction dimensions introduce communication requirements when they are partitioned.

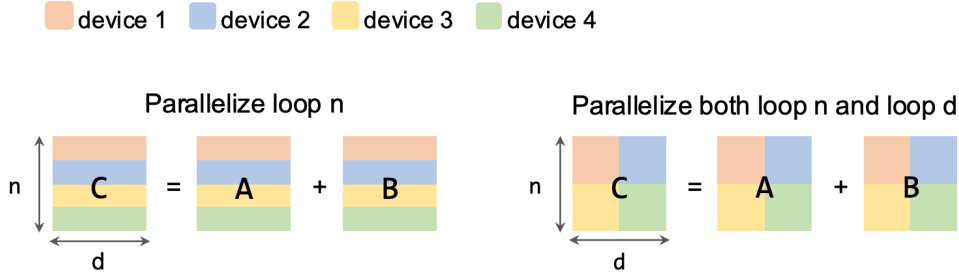


Figure 8: Element-wise operators are easy to parallelize because each output element can be computed independently. The tensor can be partitioned along one dimension or multiple dimensions.

3.3 Parallelizing Matrix Multiplication

Matrix multiplication is the central operator in modern deep learning systems. Consider

$$C = AB, \quad (3)$$

where $A \in R^{N \times K}$, $B \in R^{K \times M}$, and $C \in R^{N \times M}$. A standard loop implementation is

$$\begin{aligned} &\text{for } i = 0, \dots, N - 1 : \\ &\quad \text{for } j = 0, \dots, M - 1 : \\ &\quad \quad \text{for } k = 0, \dots, K - 1 : \\ &\quad \quad \quad C[i, j] += A[i, k] \cdot B[k, j]. \end{aligned} \quad (4)$$

This loop nest contains two different kinds of loops. The i and j loops are spatial loops because different output elements $C[i, j]$ can be computed independently. In contrast, the k loop is a reduction loop because it accumulates partial products into the same output element.

This distinction determines whether communication is required.

3.3.1 Partitioning a Spatial Dimension

First, suppose we partition the i dimension. Each device receives a different block of rows of A and computes the corresponding block of rows of C . The matrix B is replicated across devices:

$$C_r = A_r B, \quad (5)$$

where A_r and C_r denote the row shards assigned to device r .

As shown in Figure 9, this strategy does not require an all-reduce for the output, because each output element is produced by exactly one device. The main cost is that B must be available on every device. This is a common pattern: partitioning spatial dimensions often reduces computation per device without introducing reduction communication.

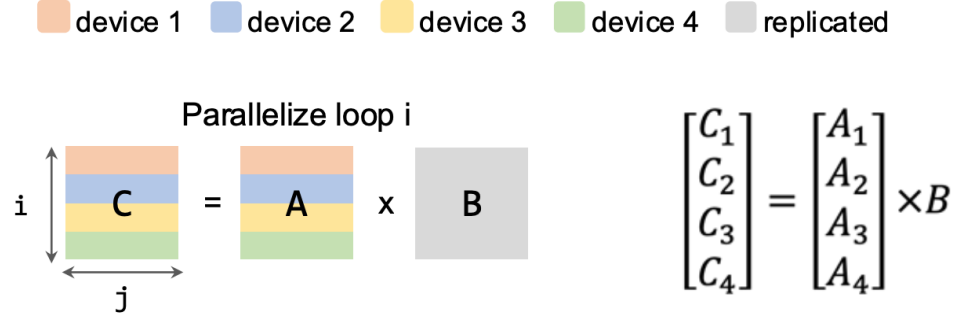


Figure 9: Partitioning matrix multiplication along a spatial loop, such as the row dimension i , lets each device compute a disjoint shard of the output. The other input matrix is replicated, and no reduction communication is required.

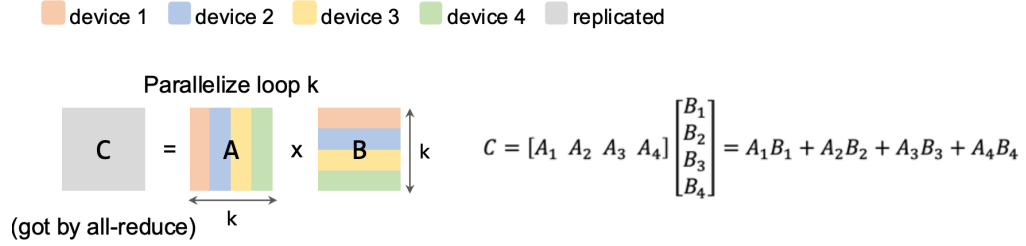


Figure 10: Partitioning matrix multiplication along the reduction dimension k creates partial outputs on different devices. These partial outputs must be summed with an all-reduce.

3.3.2 Partitioning the Reduction Dimension

Now suppose we partition the k dimension. Each device receives a shard of A 's columns and the corresponding shard of B 's rows. Device r computes a partial product

$$C^{(r)} = A^{(r)} B^{(r)}. \quad (6)$$

The final output is the sum of all partial products:

$$C = \sum_{r=1}^R C^{(r)}. \quad (7)$$

Figure 10 shows why this strategy requires communication. Each device computes only part of the sum needed for every output element. Therefore, the system must perform an all-reduce over the partial outputs to obtain the final C . This is the core reason why intra-operator parallelism is closely tied to collective communication.

3.3.3 Hybrid and Two-Dimensional Partitioning

Real systems often use more complex strategies than splitting a single dimension. For example, we can partition both i and j , producing a two-dimensional tiling of the output matrix. We can also partition i and

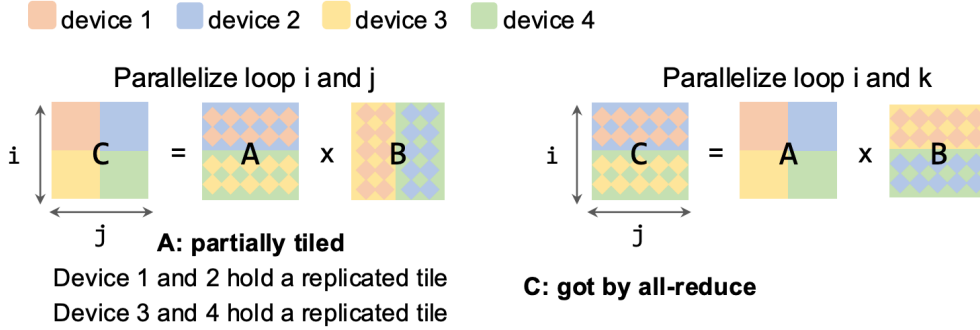


Figure 11: Matrix multiplication has many possible partitioning strategies, including two-dimensional and hybrid tilings. Different choices trade off computation balance, memory usage, and communication cost.

Partitioned dimension	Interpretation	Communication requirement
i	Row partition of A and C	Usually no output reduction
j	Column partition of B and C	Usually no output reduction
k	Reduction-dimension partition	Requires all-reduce over partial C
i, j	Two-dimensional output tiling	Depends on input layout
i, k or j, k	Hybrid spatial/reduction tiling	Requires reduction among partial outputs

Table 1: Common matrix multiplication partitioning strategies. Spatial dimensions are easier to partition, while reduction dimensions introduce collective communication.

k , in which case each device computes a partial output tile and a reduction is needed across the devices that share the same output tile.

The key lesson is that matrix multiplication has many valid parallelization strategies. Some strategies require no communication during the operator, while others require all-reduce or other collectives. Therefore, choosing the best strategy depends on both the operator shape and the hardware communication topology.

3.4 Convolution as Another Operator Example

The same reasoning applies to convolution: the batch, output channel, and spatial dimensions are spatial loops and can be freely partitioned, while the input channel dimension is a reduction loop that requires accumulation if split.

3.5 Data Parallelism as a Special Case of Intra-op Parallelism

Data parallelism can be viewed as a special case of intra-operator parallelism where operators are partitioned along the batch dimension. Each device receives a different shard of the mini-batch, runs the forward and backward passes locally, and computes local gradients.

Let θ denote the model parameters, and let $\mathcal{B}_r$ be the local mini-batch on device r . Device r computes

$$g_r = \nabla_{\theta} L(\theta; \mathcal{B}_r). \quad (8)$$

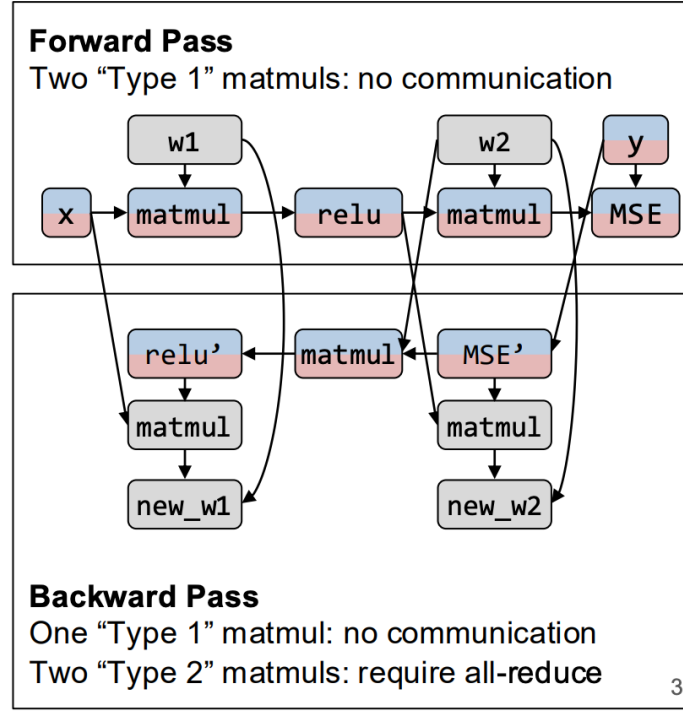


Figure 12: Data parallelism can be interpreted as partitioning operators along the batch dimension. Forward computation is mostly independent across devices, but backward computation requires gradient synchronization.

To match the semantics of training on the full global batch, the devices average their gradients:

$$g = \frac{1}{R} \sum_{r=1}^R g_r. \quad (9)$$

This averaging is typically implemented with an all-reduce.

Figure 12 shows this interpretation. During the forward pass, batch partitioning often introduces no communication because each device processes its own examples. During the backward pass, however, gradients from different devices must be synchronized. Therefore, even data parallelism can be understood through the same intra-operator lens: partitioning a dimension is easy, but preserving the correct global semantics may require communication.

3.6 Re-partition Communication Between Operators

So far, we have considered a single operator in isolation. However, a neural network is a graph of many connected operators. Even if each operator has an efficient local partitioning strategy, the entire graph may still be inefficient if adjacent operators require different tensor layouts.

For example, one matrix multiplication may output a row-partitioned tensor, while the next matrix multiplication may prefer a column-partitioned input. To connect these two operators, the system must insert a *re-partition* operation, such as all-gather or all-to-all communication.

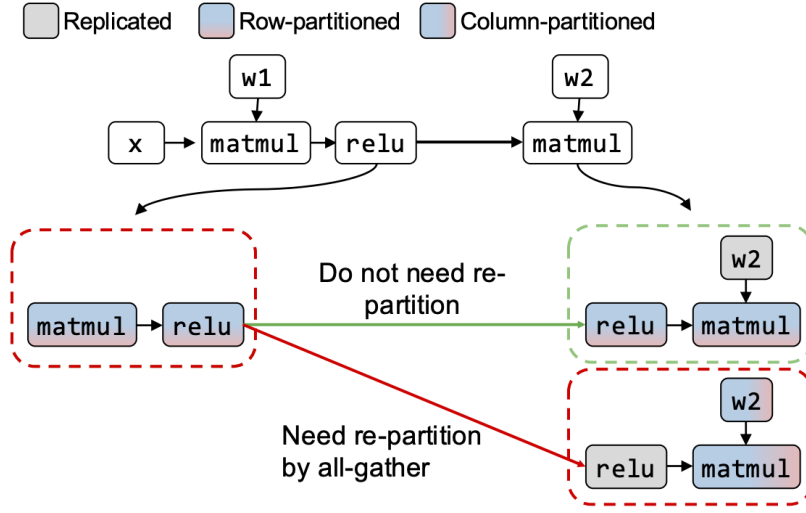


Figure 13: Different operators may prefer different tensor partition formats. When the output partition of one operator does not match the input partition required by the next operator, the system must insert re-partition communication.

Figure 13 illustrates this cost. The main point is that local optimality does not imply global optimality. A strategy that is optimal for one operator may force expensive communication before or after neighboring operators. Therefore, graph-level parallelization must account for both operator costs and edge costs.

3.7 Parallelizing the Whole Computational Graph

The full graph-level problem is to choose a parallelization strategy for every operator in the computational graph. Each strategy determines how the operator’s input and output tensors are partitioned, how much computation each device performs, and what collective communication is needed.

A useful abstraction is to model the total cost as the sum of node costs and edge costs:

$$\min_{\{s_v\}} \sum_{v \in V} \text{Cost}_{\text{node}}(v, s_v) + \sum_{(u,v) \in E} \text{Cost}_{\text{edge}}(s_u, s_v). \quad (10)$$

Here, V is the set of operators in the computational graph, E is the set of tensor dependencies between operators, and s_v is the parallelization strategy selected for operator v .

The node cost captures the cost of executing an operator under a specific strategy. It includes local computation and any collective communication required inside the operator, such as all-reduce for reduction-dimension partitioning. The edge cost captures re-partition communication between adjacent operators, such as all-gather or all-to-all when two neighboring operators require incompatible tensor layouts.

Figure 14 summarizes this optimization problem. In practice, systems use different methods to solve or approximate it, including manual design, randomized search, dynamic programming, and integer linear programming. The central challenge is balancing computation, memory, and communication across the entire graph.

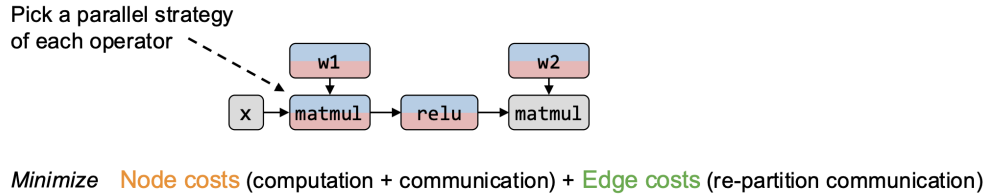


Figure 14: Graph-level parallelization chooses one strategy for each operator while minimizing both node costs and edge re-partition costs.

3.8 Short Summary

Intra-operator parallelism parallelizes the computation within a single operator by partitioning tensor dimensions and loop nests. Element-wise operators are straightforward because all output elements are independent. Matrix multiplication is more subtle because spatial dimensions can be partitioned without output reduction, while reduction dimensions require collective communication such as all-reduce.

The graph-level challenge is even harder. Each operator may have many possible parallelization strategies, and neighboring operators may prefer incompatible tensor layouts. Therefore, a distributed training system must optimize both the cost of individual operators and the re-partition communication between operators. This perspective motivates the model-specific and system-level designs discussed in the next section, including Megatron-LM, GShard, ZeRO, Mesh-TensorFlow, GSPMD, and Alpa.

4 Important Projects

The lecture ended by relating the intra-operator partitioning notation to a few well-known training systems. These examples show that the main question is not just how to split computation, but also where tensors are replicated, partitioned, and re-partitioned.

4.1 AlexNet and Megatron-LM

AlexNet is an early example of intra-operator parallelism. Its group convolution layer was split into two parts and assigned to two GPUs. This reduced per-GPU memory and computation, and the slide reports a 1.7% improvement in ImageNet top-1 accuracy. In today's notation, this is a manual partition of a convolution operator, mainly along the channel dimension.

Megatron-LM applies the same idea to Transformer models. For the MLP block

$$Y = \text{GELU}(XW_1)W_2,$$

the first matrix W_1 is column-partitioned, so each GPU computes one slice of the hidden activations. The second matrix W_2 is row-partitioned, so each GPU uses its local activation slice to produce a partial output. These partial outputs are then combined by an all-reduce. The backward pass has the corresponding all-reduce in the other direction.

The reason this works well is that the two matmuls are partitioned together. The intermediate activation can stay partitioned between them, while cheap element-wise operations such as GELU and dropout can be applied locally or replicated. This avoids unnecessary communication between the two large matrix multiplications.

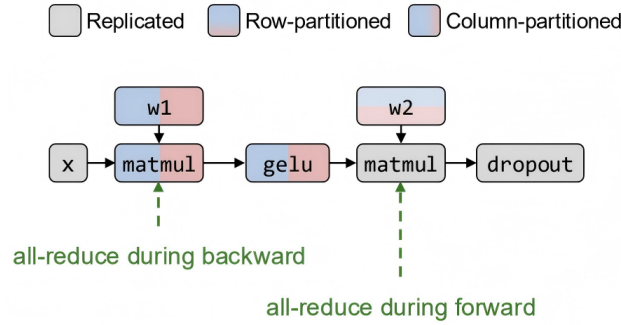


Figure 15: Megatron-LM partitions the Transformer MLP using a column-partitioned W_1 and a row-partitioned W_2 .

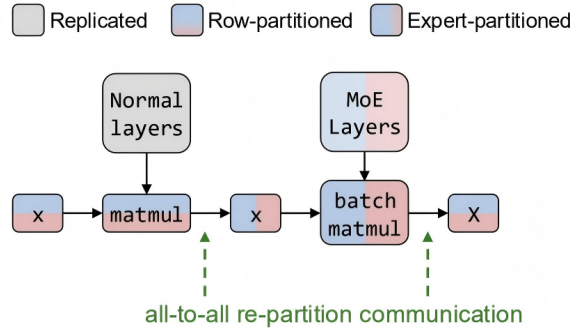


Figure 16: GShard switches between row-partitioned normal layers and expert-partitioned MoE layers using all-to-all communication.

4.2 GShard MoE

GShard uses intra-operator parallelism for Mixture-of-Experts (MoE) models. In normal Transformer layers, the lecture describes the layout as data partitioning: weights are replicated, inputs are row-partitioned, and outputs remain row-partitioned. In an MoE layer, however, experts are partitioned across devices, so the useful layout becomes expert-partitioned.

Therefore, entering the MoE layer requires an all-to-all re-partition from the batch layout to the expert layout. After the experts finish, another all-to-all switches the tensor back for the next normal layer. This is why MoE training systems care so much about efficient all-to-all kernels.

4.3 ZeRO Optimizer

ZeRO optimizes memory for data-parallel training. Standard data parallelism replicates not only model weights, but also gradients and optimizer states. With mixed-precision Adam, these auxiliary states are large: besides FP16 weights and gradients, training keeps an FP32 master copy and two FP32 Adam states.

Stage 1 shards optimizer states. Stage 2 also shards gradients by replacing an all-reduce with reduce-scatter plus all-gather, which has the same total communication cost but avoids storing full gradients on every GPU. Stage 3 also shards model weights, similar to FSDP, but must all-gather weights when forward or

Method	Optimizer states	Gradients	Weights	Memory cost	Communication
Data parallelism	Replicated	Replicated	Replicated	$16M$	all-reduce($2M$)
ZeRO Stage 1	Partitioned	Replicated	Replicated	$4M + \frac{12M}{N}$	all-reduce($2M$)
ZeRO Stage 2	Partitioned	Partitioned	Replicated	$2M + \frac{14M}{N}$	all-reduce($2M$)
ZeRO Stage 3	Partitioned	Partitioned	Partitioned	$\frac{16M}{N}$	$1.5 \times$ all-reduce($2M$)

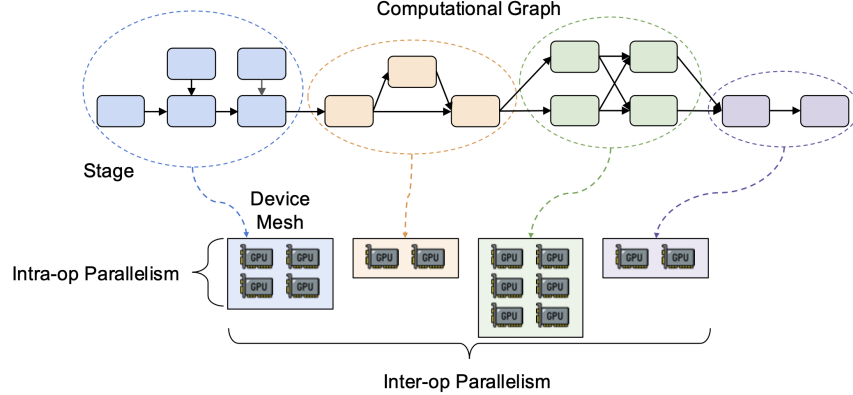
Table 2: ZeRO stages. M is the number of parameters and N is the number of devices.

Figure 17: How we combine inter- and intra-operator parallelism in today’s language models.

backward computation needs them. This saves more memory but raises the communication cost to about $1.5 \times$ all-reduce, so Stage 2 is often the better memory/communication balance unless the model is too large.

5 Combining Intra-op and Inter-op Parallelism

The professor pointed out at the end of the lecture that the two major parallelism strategies are not mutually exclusive and can be combined together to achieve optimal performance. As shown in Figure 17, the computational graph is first divided into stages (inter-op parallelism): each stage is a contiguous subgraph of operators assigned to one group of devices. Within each group, the devices form a device mesh, and the operators in that stage are further parallelized across the mesh using intra-op strategies (e.g., tensor sharding). The two dimensions are therefore hierarchical: inter-op parallelism partitions the graph along its depth, while intra-op parallelism partitions each piece along the tensor dimensions.

Experiments on GPT, GShard MoE, and Wide-ResNet show that combining the two strategies achieves higher throughput (measured in PFLOPS) than either inter-op only or intra-op only across device counts from 1 to 64 GPUs. At large device counts, single-strategy baselines also run out of memory (OOM), while the combined approach continues to scale. Therefore, the key takeaway is: *combining inter- and intra-operator parallelism scales to more devices.*

6 How to Choose a Parallelism Strategy

No single strategy is universally best. The right choice depends on model size, device count, interconnect topology, and job completion time (JCT) requirements. The professor summarized practical guidance in the

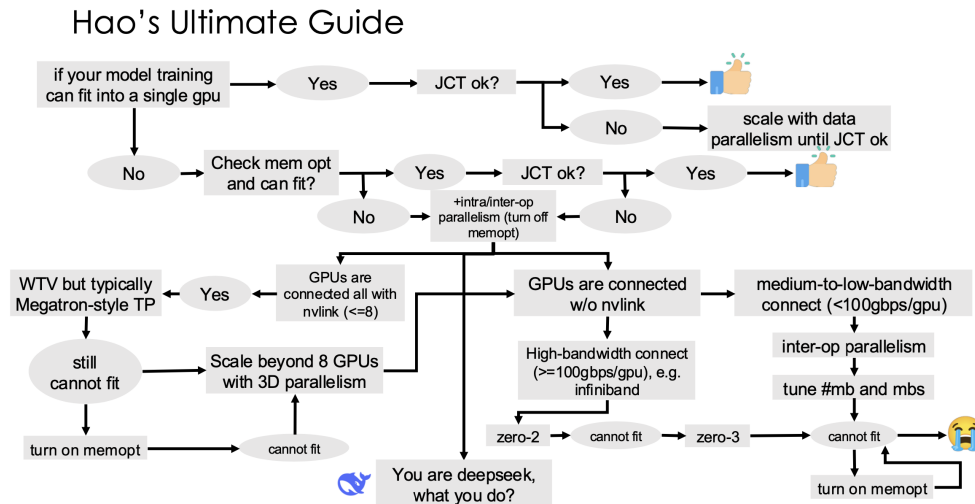


Figure 18: Hao's Ultimate Guide: A practical decision guide for parallelism strategy selection.

following decision procedure, shown in Figure 18.

1. **Model fits on a single GPU.** If JCT is acceptable, stop. Otherwise, scale with data parallelism.
2. **Model does not fit on a single GPU.** First try memory optimizations (gradient checkpointing, activation offloading). If the model now fits and JCT is acceptable, stop. Otherwise:
 - **All GPUs on NVLink (≤ 8):** Use Megatron-style tensor parallelism. Scale beyond 8 GPUs with 3D parallelism (data $\times$ tensor $\times$ pipeline).
 - **High-bandwidth inter-node (≥ 100 Gbps/GPU, e.g. InfiniBand):** Use ZeRO-2, then ZeRO-3 if still OOM.
 - **Low-to-medium bandwidth (< 100 Gbps/GPU):** Use pipeline (inter-op) parallelism and tune the number of micro-batches and micro-batch size.

7 Course Summary

This course traced the full deep learning systems stack, from dataflow graph abstraction and automatic differentiation, through graph optimization, operator implementation, and GPU architecture, to runtime memory management, quantization, LLM inference, and distributed training at scale.

The professor closed with a note on the pace of the field: individual techniques covered in this course may be superseded within one to two years. The more durable takeaway is the ability to identify the right problems, recognize trends early, and reason from first principles—since every system discussed, from register tiling to pipeline scheduling, follows from a small set of recurring principles: minimize memory traffic, maximize parallelism, overlap computation with communication, and keep communication local.